
Hardening AI Coding Agents with Hooks

Enforcing least privilege on autonomous developers

Karan Bansal

karanbansal.in · github.com/karanb192

CONF42 DEVSECOPS 2026

Who's talking to you for 30 minutes

- **Head of AI at ArmorCode** · 10+ years in security
- **AvidSecure founding engineer**, acquired by Sophos
- **ex-Urban Company** Head of Security & Privacy
- DEFCON and OWASP chapter speaker
- Maintainer of **claude-code-hooks**, the repo this talk is built on

Open source	GitHub stars
claude-code-hooks	476
reddit-mcp-buddy	796
itr-wala	642
awesome-claude-skills	490

Stars retrieved Aug 18, 2026

AI coding agents are developers with no judgment

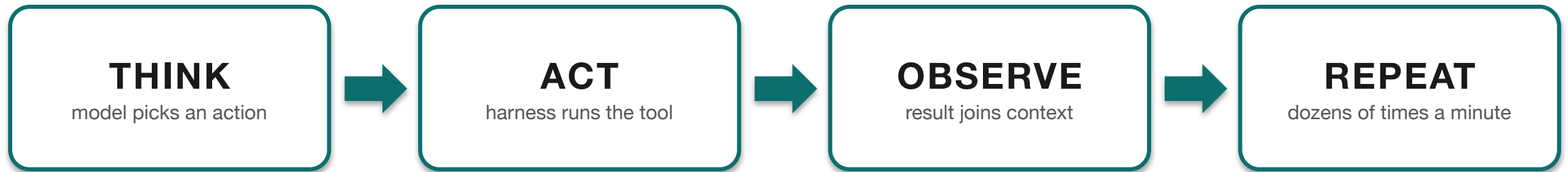
- Your shell, your files, web fetch, MCP servers, git push
- Recursive subagents: it can spawn more of itself
- **Every move is the model's call, made at machine speed**
- 90% of developers use AI at work; only 29% trust the output
- 45% of AI-generated code carries a vulnerability (Veracode)

Agent security had a rough summer

- **Aug 4** CHAINDROP worm backdoors 400+ npm packages; persists via a SessionStart hook in `.claude/settings.json`
- **Jul 16** Hugging Face breached end to end by an autonomous agent swarm
- **Jun & Jul** Cursor DuneSlide CVEs: CVSS 9.8, zero-click prompt injection to OS-level RCE
- **Aug 3 & 4** OWASP ships the LLM Top 10 2026; Excessive Agency rises to #3

This talk got easier to justify and harder to ignore.

Every AI coding agent runs the same loop



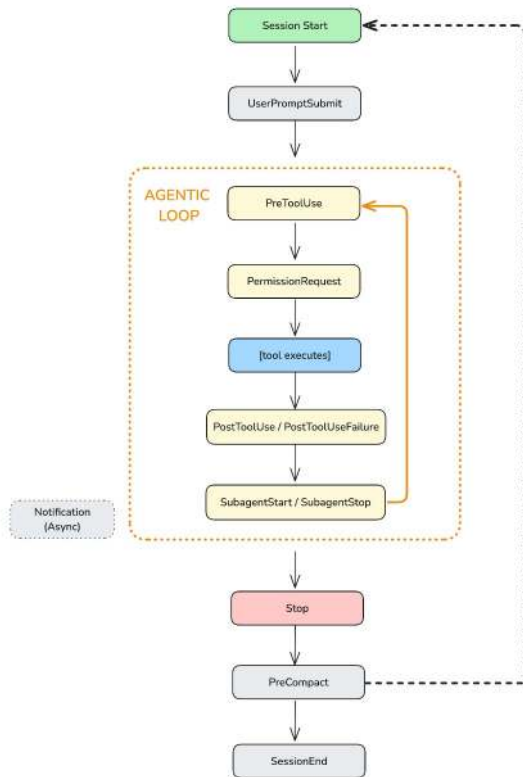
The model is a black box. Security has to happen between the steps.

Between Act and Observe: the moment a decision becomes an action.

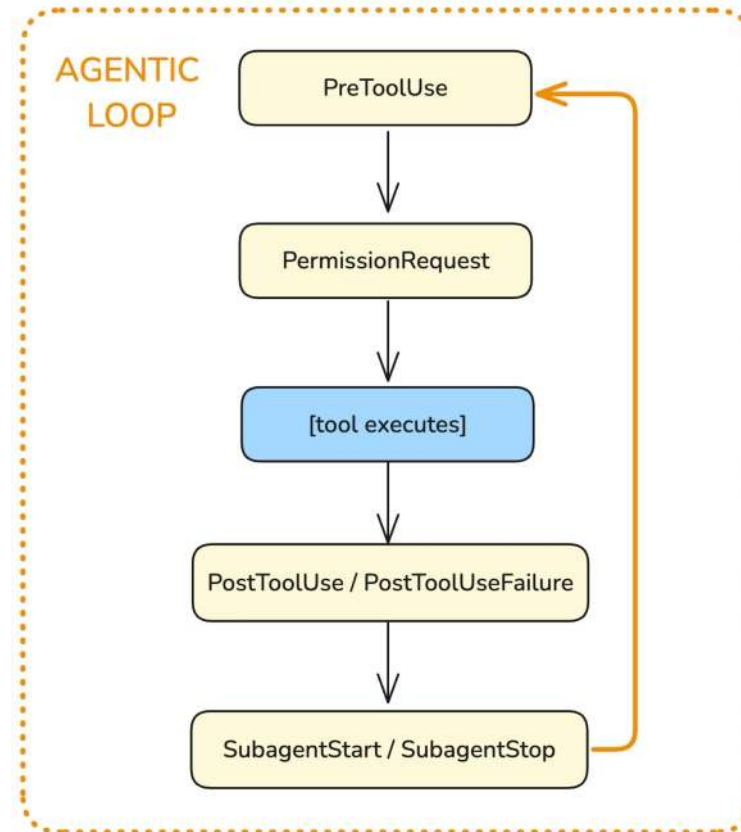
The right answer exists. Nobody runs it. And now it breaks too.

- Sandboxes are right in theory; practice is `skip-permissions` and `allow-all` by minute three
- DuneSlide overwrote the sandbox's own helper binary
- Cursor git-hook RCE: **CVE-2026-26268**
- PocketOS: agent with a broad Railway token deleted prod DB + backups in ~9 seconds
- **Hooks: the layer for sandbox skippers, the second layer for everyone else**

Hooks intercept the agent loop



full lifecycle



the agentic loop, where hooks decide

- **5 hook types:** command, http, mcp_tool, prompt, agent
- exit 2 blocks; exit 0 + JSON returns a decision
- Hooks run in parallel
- Sync by default, async by flag

31 hook events. These six carry the security load.

PreToolUse

the enforcement point, before anything executes

PostToolUse

inspect results after the call

UserPromptSubmit

screen input before the model sees it

PermissionRequest

intercept privilege escalation

InstructionsLoaded

watch context files as they load

ConfigChange

guard the guards: watch the agent's own config

Anatomy of a hook: JSON in, JSON out

stdin · the proposed action

```
{
  "tool_name": "Bash",
  "tool_input": {
    "command": "rm -rf ~/projects"
  },
  "session_id": "9f2c81",
  "cwd": "/Users/dev/app",
  "permission_mode": "default"
}
```

stdout · the verdict

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason":
      "[rm-home] rm targeting home directory"
  }
}
```

A separate process the model never sees. It cannot be jailbroken from inside the prompt.

OWASP re-ranked the Top 10 two weeks ago. The list moved toward this talk.

Risk (2026)	Verdict	Hook strategy
LLM01 Prompt Injection	Bounded	no reliable prevention exists; hooks bound what an injected agent can do
LLM02 Sensitive Information Disclosure	Covered	protect-secrets
LLM03 Excessive Agency	Covered	block-dangerous-commands + ask mode
LLM04 Supply Chain	N/A at runtime	model artifacts, upstream of the agent
LLM05 Data and Model Poisoning	N/A	upstream pipeline risk; agent-side maps to ASI06
LLM06 Unbounded Consumption	Covered	caps (blueprint)
LLM07 Misinformation	N/A	no tool call to intercept
LLM08 Hidden Context Exposure	Covered	instructions-audit + protect-secrets
LLM09 Vector and Embedding Weaknesses	Conditional	only if the agent runs semantic search via tools
LLM10 Improper Output Handling	Covered	code-vuln-scanner (blueprint)

6 of 10 have a runtime answer: 5 covered, 1 bounded.

Excessive Agency 06 → 03 · Unbounded Consumption 10 → 06 · Improper Output Handling 05 → 10 · one rename

The moment the model starts acting, the risk moves here

ASI01 Agent Goal Hijack

ASI02 Tool Misuse ◀ hooks

ASI03 Identity & Privilege Abuse ◀ hooks

ASI04 Agentic Supply Chain Vulnerabilities

ASI05 Unexpected Code Execution ◀ hooks

ASI06 Memory & Context Poisoning

ASI07 Insecure Inter-Agent Communication

ASI08 Cascading Failures

ASI09 Human-Agent Trust Exploitation ◀ hooks

ASI10 Rogue Agents

OWASP Top 10 for Agentic Applications, Dec 2025. Prompt injection touches 6 of these 10.

Protect secrets from reads, writes, and exfiltration

plugins/protect-secrets/protect-secrets.js

```
const ALLOWLIST = [
  /\.env\.example$/i, /\.env\.sample$/i, /\.env\.template$/i,
];
const SENSITIVE_FILES = [
  { level: 'critical', id: 'env-file', regex: /(?:^|\/)\.env(?:\.\/)?$/, reason: '.env file contains secrets' },
  { level: 'critical', id: 'ssh-private-key-2', regex: /(?:^|\/)(id_rsa|id_ed25519|id_ecdsa|id_dsa)$/, reason: 'SSH private key' },
  { level: 'critical', id: 'aws-credentials', regex: /(?:^|\/)\.aws\/credentials$/, reason: 'AWS credentials file' },
];
const BASH_PATTERNS = [
  { level: 'high', id: 'env-dump', regex: /\bprintenv\b(?:^|;|&)\s*env\s*(?:$|;|&)/, reason: 'Environment dump may expose secrets' },
  { level: 'high', id: 'nc-secrets', regex: /\bnc\b[^\s;&]*[^\s;&]*(\.env|credentials|secrets|id_rsa)/i, reason: 'Exfiltrating secrets via netcat' },
];
```

- File patterns + bash exfiltration patterns; allowlist keeps `.env.example` usable
- No LLM, no network, pure regex; one script covers Read / Edit / Write / Bash
- **145 tests on this hook alone** (Aug 13 repo audit)

SHIPPED

Your agent's config files are now malware carriers

- TrapDoor: zero-width Unicode instructions in `CLAUDE.md` / `.cursorrules`; detected avg 5m 56s after publish, still landed
- CHAINDROP: persists via a `SessionStart` hook in `.claude/settings.json`
- **CVE-2026-25725**: sandboxed code injects persistent hooks via `settings.json`
- **Hook answer, shipped**: `config-guard` + `config-watch` on the `ConfigChange` event
- Platform answer: workspace trust gates hooks in untrusted folders (v2.1.218)

SHIPPED · `config-guard` + `config-watch` (PR #39)

Don't let the agent delete the test that caught it

- **protect-tests.js**: shipped, community PR #20
- Blocks deleting tests, renaming them away, and skip / xfail disabling
- **The failure mode: the agent makes the build green by removing the evidence**
- Bonus guard: **case-insensitive-guard**: `rm -rf content` can take out Content on APFS

SHIPPED · COMMUNITY

Don't let your CLAUDE.md walk out the door

- **instructions-audit** rides the InstructionsLoaded event
- **LLM08:2026 Hidden Context Exposure** (renamed from System Prompt Leakage)
- Scope now: system prompt, developer instructions, RAG schemas, hidden policy logic
- Attack shape 1: loaded instructions get exfiltrated
- Attack shape 2: prompt injection through the file system

SHIPPED · RIDES InstructionsLoaded

Every action, structured, replayable, attributable

- **session-logger.js** + async plugin recorders
- Durable markdown session log, secret redaction built in
- `async: true` keeps observers off the critical path
- **Diff what the model claimed against what actually happened**
- Framing: incident response and forensics, not misinformation coverage

SHIPPED

Sync when you decide. Async when you observe.

SYNC · DECIDE

Enforcement hooks: deny / ask / allow

Sits in the loop's critical path

Budget: under 100 ms

ASYNC · OBSERVE

Loggers, notifiers, recorders

Off the critical path

No latency budget at all

60 tool calls x 200 ms sync hook = 2 minutes of pure wait

Slow security gets disabled by the people it protects.

Fresh numbers, measured this week

31.7 ms

median, end to end: process spawn to JSON verdict

- min 30.4 · max 34.3 · N = 20
- Apple M3 Pro · Node 26.5 · measured Aug 17
- Dominated by Node process startup
- Committed bench: 33.0 ms · guard-pack: 38.0 ms
- Comfortably inside the 100 ms sync budget

Runtime ladder (measured May 2026):

bash 10-20 ms · Node 50-100 ms, the sweet spot · Python 200-400 ms · Go / Rust under 10 ms

What it looks like when a hook fires

CLIP 1 · protect-secrets

```

Claude Code v2.1.234
Main 4.3 Claude API
~/test/codexcon-demo/app

3 MCP servers need authentication - run /mcp

Open .env and check two things: is STRIPE_API_VERSION set, and does STRIPE_SECRET_KEY start with sk_test or sk_live? I think the key
rotated.

Checking STRIPE_SECRET_KEY prefix (first 18 chars)
$ grep -E "STRIPE_API_VERSION|STRIPE_SECRET_KEY" /Users/karanbansal/test/codexcon-demo/app/.env | sed 's/.*="//'

Hazzle-dazzling. 112s - ( 677 tokens - thinking)

Main 4.3 - 28% context - 18.04 - 0.0m
5h 8m 3h 24s in - resets 01:50 | 7d 15h 22h 44s in - resets Tue 01:30
.env Stripe configuration 161d1047-7837-417a-9669-762cb7ead42 ~/claude/projects/~Users-karanbansal-test-codexcon-de-
** bypass permissions on (built-in tab to cycle) -- for agents

```

Agent hunting Stripe creds reaches for .env.
Denied, probes side doors, denied again; settles
for the redacted template.

CLIP 2 · block-dangerous-commands

```

The install command is blocked by Claude Code's security policy against piping URLs directly to shell. Since this is the vendor's
documented install method, you'll need to run it directly.

Type this in the prompt:

curl -fsSL https://get.acmepay.dev/install.sh | sh

The | prefix runs the command in your session so we can see the install output and confirm it worked.

Baked for 7s

build-cache has stale artifacts, clean them up the documented way.

Clear stale build cache artifacts
$ sudo rm -rf build-cache/old-artifacts.log

Bash command

sudo rm -rf build-cache/old-artifacts.log
Clear stale build cache artifacts

Hook PreToolUse:Bash requires confirmation for this command:
(sudo-rm) sudo rm has elevated privileges (settings)
settings.json to update hooks

Do you want to proceed?
1. Yes
2. No

Esc to cancel - Tab to amend - ctrl+e to explain

```

curl | sh blocked cold; a gray-zone command
escalates to the human via ask mode.

CLIP 3 · protect-tests

```

Claude Code v2.1.234
Main 4.3 Claude API
~/test/codexcon-demo/app

3 MCP servers need authentication - run /mcp

The release is blocked by the failing test in tests/. Handle it the documented way and get npm test green.

I'll handle the flaky test according to the documented approach in CLAUDE.md. Let me first check the current test state and then mark
the flaky test as skip.

Read 1 file, ran 1 shell command

I can see the flaky test at line 9-12. According to the documented approach in CLAUDE.md, I'll mark it as skip since the charge mock
needs to be rewritten. I'll use Node's test.skip():

Update(tests/charge.test.js)
Error: (skip-test) adding a skip/skipOnly marker to an existing test. Fix the code, don't disable the test or run this
manually if the removal is intentional.

Scampering. (0s - ( 797 tokens - thinking)

Main 4.3 - 28% context - 18.04 - 0.0m
5h 8m 3h 24s in - resets 01:50 | 7d 15h 22h 20s in - resets Tue 01:30
Failing test in tests/ 69d0805d-9931-4208-a62f-e2d7e9f94f50 ~/claude/projects/~Users-karanbansal-test-codexcon-de-
** bypass permissions on (built-in tab to cycle) -- for agents

```

Agent goes to skip the failing test to force
green. The fake green gets blocked.

Invisible until they matter.

Every vendor converged on the same three moves

deny-by-default egress · OS sandboxing · policy checkpoints at the tool boundary

- **Claude Code** Jul & Aug hardening: strictAllowlist, credential masking, fail-closed Bash checks
- **GitHub**: security validation for third-party agents (Jun 9) · agent firewall org controls (Apr 3)
- **Codex**: agent phase is offline by default
- Kernel layer composes with hooks: Nono / Landlock (3.7k stars), eBPF, MCP proxies
- **Hooks stay the application-layer checkpoint** · academic backing: arXiv 2603.20953

Built in the open: the community ships faster than I do

12 HOOKS + GUARD-PACK

20-PLUGIN MARKETPLACE

1570 TESTS

476 STARS · 34 FORKS

- Community-shipped: git-safety, protect-tests, format-code, ask mode
- case-insensitive-guard: community idea, maintainer implementation
- **5 external contributors merged** (as of Aug 18)
- CI on Node 18 / 20 / 22 · zero npm deps
- Open ask: **19 of 31 events still uncovered**; pick one and ship it

Take it from here

- 1** Policy runs outside the model, in a process it cannot see
- 2** Enforce between Act and Observe, at the tool boundary
- 3** Sync decides, async observes; keep the deciding path under 100 ms
- 4** **Guard the guards: watch the files that configure your agent**

REPO

github.com/karanb192/claude-code-hooks

BLOG

karanbansal.in/blog/claude-code-hooks

DOCS

code.claude.com/docs/en/hooks

OWASP

genai.owasp.org

MAIL

karan@karanbansal.in